Projektreflexion – Phase 2

Inhaltsverzeichnis

Umsetzung des Konzepts 2

Verwendete Technologien 2

Technische Herausforderungen 4

Literaturverzeichnis 6

Umsetzung des Konzepts

Das Hauptziel war, dass Kunden Nagellacke ansehen, Produkte in den Warenkorb legen und Bestellungen aufgeben können. Außerdem wollte ich einen Kontobereich entwickeln, in dem Adressen verwaltet und Bestellungen eingesehen werden können. Für Administratoren sollten separate Seiten zur Verwaltung von Produkten und Kategorien zur Verfügung stehen.

Ich wollte erst einmal das Backend für jede Seite fertigstellen und habe mich mehr auf die Funktionen als auf das Design konzentriert. Es war sehr wichtig, dass die App fehlerfrei auf Benutzereingaben reagiert und mit der Datenbank zusammenarbeitet

Also deswegen, habe ich zu Beginn die Anwendung lokal auf meinem Mac entwickelt und an den einzelnen Funktionen und Seiten gearbeitet. Ich habe Java 17, Maven und MySQL installiert. Vs Code hatte ich aufgrund früherer Erfahrungen bereits installiert. Ich habe trotzdem Extensions wie “Java” und “Prettier” installiert, um mir bei der Entwicklung zu helfen. Maven hat die Projekt-dependencies für mich installiert. Docker und Colima kamen erst später hinzu, als etwa zwei Drittel des Fortschritts bereits abgeschlossen waren.

Für die Bezahlung habe ich mich für Stripe entschieden, weil Kunden damit zum Beispiel mit ihrer eigenen Karte oder über Klarna bezahlen können. Auch die Umsetzung fand ich spannend: Man kann eine API einbinden, und die Arbeit mit der Sandbox fand ich hilfreich für einen späteren Einsatz. Zum Testen habe ich den Stripe-Testmodus integriert, um den Zahlungsvorgang ohne echte Transaktionen überprüfen zu können (vgl. Stripe, o. J.).

Am Anfang fand ich eine DE/EN-Umschaltung eine tolle Idee. Leider habe ich den Aufwand und die verfügbare Zeit nicht richtig eingeschätzt. Zuerst habe ich alles auf Englisch programmiert und später entschieden, zunächst nur die deutsche Version anzubieten. Vielleicht setze ich diese Funktion in Zukunft noch um. Ich fand sie für internationale Bestellungen wirklich sinnvoll.

Verwendete Technologien

Die Anwendung verwendet Java 17. Mit Java habe ich die Anwendungslogik entwickelt und Produkte, Kunden und Bestellungen in verschiedenen Klassen organisiert. Spring Boot habe ich genutzt, um die Webanwendung zu strukturieren. Der Code ist in Controller, Services, Repositories und Entity-Klassen aufgeteilt. Bei der Arbeit mit dieser Struktur habe ich besser verstanden, welche Aufgabe zu welchem Teil gehört. Dadurch konnte ich leichter nachvollziehen, wo ich eine Funktion bearbeiten musste.

Ein Beispiel dafür ist die Konstruktorinjektion. Der ProductController erhält die benötigten Repositories über seinen Konstruktor. Ich musste diese Objekte dadurch nicht im

Controller selbst erstellen. Spring stellt die benötigten Abhängigkeiten beim Erstellen des Objekts bereit (vgl. Spring, o. J.-a).

Bei der Anmeldung habe ich mit UserDetailsService gearbeitet. UserDetailsService stellt Spring Security die Benutzerdaten für die Anmeldung bereit (vgl. Spring, o. J.-b). Dadurch habe ich besser verstanden, wie die eigenen Kundendaten mit der Anmeldung verbunden werden.

Auch @Override wurde durch dieses Beispiel verständlicher: Die eigene Methode implementiert die Vorgabe der Schnittstelle. Die Annotation lässt den Compiler prüfen, ob die Methode tatsächlich eine passende Methode implementiert oder überschreibt (vgl. Oracle, o. J.).

Für die Produktseiten habe ich Thymeleaf verwendet. Die Namen, Preise und Lagerbestände sollten aus der Datenbank kommen, statt noch einmal fest in den HTML-Seiten zu stehen. Thymeleaf kann Werte aus dem Datenmodell in HTML-Vorlagen einsetzen (vgl. Thymeleaf, o. J.).

JavaScript wollte ich nur verwenden, wenn es wirklich nötig war. Zum Beispiel sollte ein Kunde ein Produkt in den Warenkorb legen können, ohne die Seite zu verlassen oder seine Position auf der Seite zu verlieren. Für andere Funktionen waren normale Links und Formulare ausreichend.

MySQL habe ich verwendet, um die Produkte, Kategorien, Kunden, Adressen und Bestellungen zu speichern. Diese Informationen müssen erhalten bleiben und dürfen nicht nur während der Ausführung der Anwendung vorhanden sein.

Für das Erstellen der Anwendung und die benötigten Bibliotheken habe ich Maven genutzt. Dadurch musste ich nicht jede Bibliothek einzeln herunterladen. Die Abhängigkeiten werden in der pom.xml angegeben und von Maven aufgelöst (vgl. Apache Maven, o. J.). Im Projekt gehören dazu unter anderem Spring Boot, Thymeleaf, Spring Security, der MySQL-Treiber und die Stripe-Bibliothek. Als Entwicklungsumgebung habe ich VS Code verwendet. Im integrierten Terminal konnte ich Befehle ausführen und Fehlermeldungen direkt beim Arbeiten untersuchen.

Später habe ich Docker verwendet, um zu prüfen, ob die Anwendung auch unabhängig von meinem Mac funktioniert, weil der Tutor sollte eine vergleichbare Umgebung starten können. Docker Compose beschreibt die Dienste und ihre Verbindungen in einer gemeinsamen Konfiguration (vgl. Docker, o. J.). Da ich macOS verwende, habe ich dafür auch Colima kennengelernt. Colima ermöglicht den Betrieb von Containern auf dem Mac (vgl. Colima, o. J.). Das war meine erste Erfahrung mit diesem Aufbau. Dabei habe ich gelernt, dass neben dem Anwendungscode auch die Laufzeitumgebung vorbereitet werden muss.

Technische Herausforderungen

Nachdem die Anwendung lokal auf meinem Mac funktionierte, habe ich mich gefragt, ob der Tutor sie auch auf seinem Computer starten könnte. Deshalb reichte es nicht, nur lokal zu testen. Ich entschied mich für eine Docker-Umgebung, um die Anwendung unter vergleichbaren Bedingungen ausprobieren und weitergeben zu können. Ein Container führt eine Anwendung in einer isolierten Umgebung mit den benötigten Dateien und Bibliotheken aus. Eine virtuelle Maschine enthält dagegen ein eigenes Betriebssystem (vgl. Docker, o.J.).

Eine der ersten Schwierigkeiten war, dass die neue Datenbank im Docker-Container meine bisherigen Daten nicht enthielt. Die Anwendung konnte starten, aber die Produkte waren nicht da. Deshalb musste ich die alte Datenbank exportieren und in die neue Datenbank importieren. Dabei habe ich gemerkt, dass eine laufende Anwendung noch nicht bedeutet, dass auch alle benötigten Daten vorhanden sind. Auch die Verbindung zur Datenbank war am Anfang nicht einfach. Danach konnte ich besser nachvollziehen, warum eine Einstellung auf meinem Mac funktionieren konnte, aber im Container angepasst werden musste.

Eine weitere Schwierigkeit war, die Anwendung über localhost:8080 im Browser meines Macs zu erreichen. Dafür musste ich zuerst besser verstehen, was Colima eigentlich macht. In meinem Aufbau stellt Colima eine Linux-VM bereit, in der Docker und die Container laufen. Diese virtuelle Maschine ist eine eigene Umgebung innerhalb meines Macs und von macOS getrennt (vgl. Colima, o. J.). Die Anwendung lief also nicht direkt auf dem Mac. Damit mein Browser sie erreichen konnte, musste die Verbindung vom Mac über die Portweiterleitung zur VM und zum App-Container funktionieren. Zusätzlich lief auf meinem Mac noch eine ältere Anwendung auf Port 8080. Dadurch passten die Ergebnisse im Browser zunächst nicht zu der Datenbank, die ich gerade überprüfte. Später war die Container-Anwendung vom Mac aus nicht erreichbar, obwohl sie innerhalb der Container-Umgebung antwortete. Das war verwirrend, weil es zuerst wie ein Fehler im Code aussah. Nach dem Beenden der alten Anwendung und einem Neustart von Colima konnte ich die Container-Anwendung im Browser öffnen. Dabei habe ich gelernt, dass eine laufende App im Container nicht automatisch bedeutet, dass sie auch vom Mac aus erreichbar ist.

Später habe ich festgestellt, dass das Hinzufügen eines Produkts mit leeren Feldern zu einem Anwendungsfehler führte. Deshalb wurden Name, Preis und Lagerbestand im Formular zu Pflichtfeldern gemacht und zusätzlich in Java überprüft. Ich wollte verhindern, dass unvollständige Produkte gespeichert werden und die Anwendung dadurch einen Fehler zeigt.

Für die Weitergabe an den Tutor kam noch eine Schwierigkeit dazu. Eine Kopie des Projekts enthielt nicht automatisch die Daten aus meiner laufenden Datenbank. Ich wollte aber, dass der Tutor direkt Produkte sehen und die Konten und Bestellungen ausprobieren kann. Deshalb wurde eine SQL-Datei zur automatischen Initialisierung hinzugefügt. Das offizielle MySQL-Docker-Image führt solche SQL-Dateien beim Einrichten einer neuen, leeren Datenbank aus (vgl. MySQL Docker Official Image, o. J.). In dieser Datei sind die Produkte, Kategorien, Rabatte, Lagerbestände und Bildverweise enthalten. Dazu kommen

zwei Demo-Konten sowie fiktive Adressen und Beispielbestellungen. Andere Kundendaten wurden nicht übernommen.

Persönliche Reflexion

Es gibt noch einiges zu tun, besonders beim Design. Trotzdem bin ich glücklich, dass die wichtigsten Funktionen der App gut laufen. Produkte können angezeigt, in den Warenkorb gelegt und bestellt werden. Auch die Kontos und die Bezahlung funktionieren.

Ich bin auch glücklich, dass ich Java und MySQL aus anderen Kursen anwenden konnte. Dadurch habe ich besser verstanden, wie die verschiedenen Teile zusammenarbeiten.

Besonders freue ich mich, dass ich gelernt habe, mit Docker zu arbeiten. Das war meine erste Erfahrung damit. Diese Fähigkeit wollte ich auch für mein späteres Berufsleben lernen und mit diesem Projekt konnte ich sie direkt ausprobieren.

Wenn ich noch einmal anfangen würde, würde ich direkt in einer Container-Umgebung entwickeln. So ist meiner Meinung nach sicherer, vor allem, weil viele Abhängigkeiten herunterladen werden sollen (ehrlich gesagt sind es so viele, dass sich Malware oder schädliche Software sehr leicht auf meinen eigenen Rechner einschleichen könnten ohne zu wissen).

Ich vermute, dass der Aufbau auf einem Linux-Rechner einfacher gewesen wäre. Dann hätte ich Colima nicht gebraucht.

Ich glaube, ich habe etwa einen halben Tag damit verbracht, herauszufinden, warum die Anwendung vom Container aus nicht auf localhost angezeigt wird. Ich dachte, das müsste doch einfach funktionieren. Später habe ich verstanden, dass mein Mac, die VM, die Container und die Portweiterleitung zusammenarbeiten müssen.

Als Nächstes möchte ich das Design verbessern. Die App ist noch nicht in jedem Detail fertig. Trotzdem bin ich mit dem bisherigen Ergebnis zufrieden.

Literaturverzeichnis

Apache Maven (o. J.): *Introduction to the Dependency Mechanism*. Verfügbar unter: https://maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html (Zugriff: 14.10.2026).

Colima (o. J.): *Colima – Container Runtimes on macOS and Linux*. Verfügbar unter: https://colima.run/ (Zugriff: 14.10.2026).

Docker (o. J.-a): *Compose Application Model*. Verfügbar unter: https://docs.docker.com/compose/intro/compose-application-model/ (Zugriff: 14.10.2026).

Docker (o. J.-b): *What is a Container?* Verfügbar unter: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/ (Zugriff: 14.10.2026).

MySQL Docker Official Image (o. J.): *MySQL – Docker Official Image*. Verfügbar unter: https://hub.docker.com/_/mysql (Zugriff: 14.10.2026).

Oracle (o. J.): *Annotation Interface Override – Java SE 17*. Verfügbar unter: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Override.html (Zugriff: 14.10.2026).

Spring (o. J.-a): *Dependency Injection*. Verfügbar unter: https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html (Zugriff: 14.10.2026).

Spring (o. J.-b): *UserDetailsService*. Verfügbar unter: https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/user-details-service.html (Zugriff: 14.10.2026).

Stripe (o. J.): *Stripe Checkout*. Verfügbar unter: https://docs.stripe.com/payments/checkout (Zugriff: 14.10.2026).

Thymeleaf (o. J.): *Using Thymeleaf*. Verfügbar unter: https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html (Zugriff: 14.10.2026).